

Language Models and RNNs

Nguyễn Duy Tân *

EE HCMUT ML&IoT Lab,
Trường Đại học Bách khoa Thành phố Hồ Chí Minh,
Đại học Quốc gia Thành phố Hồ Chí Minh
`tan.nguyen231305@hcmut.edu.vn`

Thành phố Hồ Chí Minh, Tháng 7 Năm 2026

Abstract

Nội dung của tài liệu tập trung vào bài toán Mô hình Ngôn ngữ (Language Modeling) và sự tiến hóa từ mô hình n-gram truyền thống sang các Mạng nơ-ron Hồi quy (RNN). Do mô hình n-gram gặp phải vấn đề nghiêm trọng về tính thưa thớt dữ liệu (sparsity) và không gian lưu trữ, RNN đã ra đời nhằm khắc phục những hạn chế này cũng như giới hạn của mô hình nơ-ron sử dụng cửa sổ cố định bằng cách dùng chung một bộ ma trận trọng số qua từng bước thời gian, cho phép hệ thống xử lý các chuỗi văn bản có độ dài bất kỳ. Các mô hình này được huấn luyện bằng thuật toán lan truyền ngược theo thời gian để tối thiểu hóa hàm mất mát cross-entropy và được đánh giá hiệu năng thông qua chỉ số độ rối (Perplexity). Tuy nhiên, thực tế huấn luyện RNN cơ bản gặp phải rào cản lớn do hiện tượng bùng nổ và biến mất đạo hàm (exploding and vanishing gradients), khiến mô hình khó học được các phụ thuộc ngôn ngữ ở khoảng cách xa. Để giải quyết các điểm yếu này, các kỹ thuật như cắt tỉa gradient (gradient clipping) được áp dụng, đồng thời các cấu trúc nơ-ron phức tạp hơn như GRU và LSTM ra đời với cơ chế cổng (gates) giúp duy trì và kiểm soát bộ nhớ dài hạn một cách hiệu quả hơn. Cuối cùng, tài liệu mở rộng việc áp dụng các mạng RNN (bao gồm cả RNN đa chiều và mạng sâu) vào các tác vụ Dịch máy Nơ-ron (NMT) thông qua kiến trúc Sequence-to-Sequence (Encoder-Decoder), tạo tiền đề giải quyết nhiều vấn đề NLP phức tạp dù vẫn còn tồn tại một số thách thức về nút thắt cổ chai (bottleneck) thông tin khi mã hóa câu dài.

1 Giới thiệu chung và Khái niệm Cốt lõi: Mô hình Ngôn ngữ

Mô hình Ngôn ngữ (Language Model – LM) là một trong những khái niệm quan trọng nhất trong Xử lý Ngôn ngữ Tự nhiên hiện đại, đóng vai trò nền tảng cho hầu hết các hệ thống trí tuệ nhân tạo tạo sinh văn bản ngày nay [?, p.2]. Về bản chất, một mô hình ngôn ngữ là một hệ thống học cách dự đoán từ tiếp theo trong một chuỗi văn bản, dựa trên những từ đã xuất hiện trước đó. Phần này sẽ trình bày định nghĩa hình thức của bài toán, tầm quan trọng của nó trong các ứng dụng thực tế, cũng như giới thiệu các mô hình n-gram truyền thống và những hạn chế cố hữu dẫn đến sự ra đời của các mô hình ngôn ngữ dựa trên mạng nơ-ron.

1.1 Định nghĩa và Nhiệm vụ của Mô hình Ngôn ngữ

Cho một chuỗi các từ $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, nhiệm vụ của mô hình ngôn ngữ là tính toán phân phối xác suất của từ tiếp theo $x^{(t+1)}$ trong từ điển V [?, p.3]:

$$P(x^{(t+1)} | x^{(1)}, \dots, x^{(t)}),$$

* Bài viết được viết lại từ các slides [?, ?] và sử dụng AI như một công cụ hỗ trợ.

trong đó $x^{(t+1)}$ có thể là bất kỳ từ nào trong tập từ vựng $V = \{w_1, \dots, w_{|V|}\}$. Nói cách khác, mô hình ngôn ngữ cung cấp một cách để đánh giá xem một từ nào đó có khả năng xuất hiện tiếp theo cao đến mức nào dựa trên ngữ cảnh đã thấy.

Mở rộng ra, một mô hình ngôn ngữ cũng có thể được xem như một hệ thống gán xác suất cho toàn bộ một đoạn văn bản. Xác suất của một chuỗi văn bản $x^{(1)}, \dots, x^{(T)}$ được tính bằng tích của các xác suất có điều kiện của từng từ, theo quy tắc chuỗi (chain rule) trong xác suất [?, p.4]:

$$P(x^{(1)}, \dots, x^{(T)}) = \prod_{t=1}^T P(x^{(t)} \mid x^{(t-1)}, \dots, x^{(1)}).$$

Đây là một cách biểu diễn xác suất rất mạnh mẽ, cho phép các mô hình ngôn ngữ được sử dụng để so sánh tính "tự nhiên" của các câu văn khác nhau.

1.2 Tầm quan trọng và Ứng dụng của Mô hình Ngôn ngữ

Mô hình ngôn ngữ không chỉ là một bài toán học thuật mà còn là một thành phần cốt lõi trong vô số ứng dụng thực tế [?, p.5]. Mỗi lần chúng ta gõ tin nhắn và thấy gợi ý từ tiếp theo, đó là một mô hình ngôn ngữ đang hoạt động. Một số ứng dụng tiêu biểu bao gồm:

- **Gõ văn bản dự đoán (Predictive typing):** Dự đoán từ tiếp theo để tăng tốc độ nhập liệu trên điện thoại thông minh.
- **Nhận dạng giọng nói (Speech recognition):** Đánh giá khả năng của các chuỗi từ ứng viên để chọn ra bản phiên âm chính xác nhất.
- **Sửa lỗi chính tả và ngữ pháp:** Xác định và sửa các lỗi dựa trên ngữ cảnh xung quanh.
- **Dịch máy (Machine Translation):** Đánh giá xem một bản dịch có tự nhiên và mạch lạc hay không.
- **Tóm tắt văn bản (Summarization) và Đối thoại (Dialogue):** Sinh ra các văn bản mới một cách trôi chảy và phù hợp.

Quan trọng nhất, mọi hệ thống trí tuệ nhân tạo tạo sinh văn bản hiện đại, bao gồm cả ChatGPT, về bản chất đều là những mô hình ngôn ngữ được huấn luyện trên quy mô cực lớn [?, p.7]. Khả năng dự đoán từ tiếp theo, tưởng chừng đơn giản, đã chứng minh sức mạnh phi thường khi có đủ dữ liệu và sức tính toán, cho phép mô hình thực hiện các tác vụ phức tạp như trả lời câu hỏi, suy luận, và thậm chí cả phân tích cảm xúc [?, p.8].

1.3 Mô hình ngôn ngữ n-gram: Tiếp cận truyền thống

Trước kỷ nguyên học sâu, cách tiếp cận phổ biến nhất để xây dựng mô hình ngôn ngữ là sử dụng các mô hình **n-gram** [?, p.9]. Một n-gram là một chuỗi gồm n từ liên tiếp nhau (ví dụ: unigram cho 1 từ, bigram cho 2 từ, trigram cho 3 từ). Ý tưởng cốt lõi là thu thập thống kê về tần suất xuất hiện của các n-gram trong một kho ngữ liệu lớn và sử dụng những thống kê này để ước lượng xác suất.

Mô hình n-gram dựa trên một **giả định Markov**: từ tiếp theo $x^{(t+1)}$ chỉ phụ thuộc vào $n - 1$ từ trước đó, thay vì toàn bộ lịch sử [?, p.10]. Cụ thể, xác suất được xấp xỉ bằng tần suất tương đối:

$$P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(t-n+2)}) \approx \frac{\text{count}(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)})}{\text{count}(x^{(t)}, \dots, x^{(t-n+2)})}.$$

Ví dụ, với mô hình trigram (dựa trên 2 từ trước đó), xác suất của từ w_3 khi biết w_1, w_2 được tính là [?, p.2]:

$$P(w_3 \mid w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}.$$

Phương pháp này đơn giản và hiệu quả cho các ngữ cảnh ngắn, nhưng gặp phải ba vấn đề nghiêm trọng [?, p.2]:

1. **Tính thưa thớt (Sparsity):** Nếu một n-gram chưa từng xuất hiện trong tập huấn luyện, xác suất của nó sẽ bằng 0. Điều này thường xuyên xảy ra với các n-gram dài, ngay cả trên ngữ liệu lớn. Các kỹ thuật như làm mịn (smoothing – thêm một hằng số nhỏ vào các giá trị đếm) hoặc lùi lại (backoff – giảm kích thước ngữ cảnh nếu không có đủ dữ liệu) có thể giúp giảm thiểu nhưng không giải quyết triệt để.
2. **Yêu cầu lưu trữ lớn:** Mô hình phải lưu trữ số đếm của tất cả các n-gram đã thấy. Khi n tăng hoặc kích thước ngữ liệu tăng, bộ nhớ cần thiết tăng theo cấp số nhân, khiến việc sử dụng $n > 5$ trở nên không khả thi [?, p.12].
3. **Thiếu khả năng gắn kết toàn cục:** Vì chỉ nhìn vào một cửa sổ ngữ cảnh cố định và ngắn, các văn bản được sinh ra bởi mô hình n-gram có thể đúng ngữ pháp nhưng thường không mạch lạc và thiếu tính liên kết về ý nghĩa [?, p.18].

1.4 Mô hình ngôn ngữ nơ-ron với cửa sổ cố định

Mô hình ngôn ngữ nơ-ron đầu tiên được Bengio và cộng sự giới thiệu năm 2000/2003 [?, p.22] đã giải quyết được vấn đề thưa thớt bằng cách học một biểu diễn phân bố (distributed representation) của từ vựng, nhưng vẫn bị giới hạn bởi một cửa sổ ngữ cảnh cố định. Mô hình này [?, p.3] sử dụng các word embeddings (được học cùng lúc) cho một cửa sổ n từ, ghép chúng lại, và đưa qua một hoặc nhiều tầng ẩn với phi tuyến tanh trước khi đưa vào softmax để dự đoán từ tiếp theo.

Mặc dù khắc phục được vấn đề thưa thớt và lưu trữ, mô hình cửa sổ cố định có hai hạn chế lớn [?, p.22]:

- **Không thể xử lý ngữ cảnh dài tùy ý:** Việc tăng kích thước cửa sổ sẽ làm tăng ma trận trọng số W và không bao giờ đủ lớn để bao quát toàn bộ ngữ cảnh.
- **Thiếu tính đối xứng:** Các từ ở các vị trí khác nhau trong cửa sổ được nhân với các ma trận trọng số khác nhau, dẫn đến việc mô hình không xử lý các vị trí một cách nhất quán.

Những hạn chế này đặt ra nhu cầu cho một kiến trúc có thể xử lý đầu vào có độ dài biến đổi và chia sẻ trọng số một cách đối xứng – đó chính là lý do cho sự ra đời của **Mạng nơ-ron Hồi quy (Recurrent Neural Networks – RNN)**, sẽ được trình bày chi tiết trong phần tiếp theo.

2 Mạng nơ-ron Hồi quy (Recurrent Neural Networks)

Phần trước đã chỉ ra những hạn chế cố hữu của mô hình ngôn ngữ n-gram và mô hình nơ-ron cửa sổ cố định: không thể mô hình hóa các phụ thuộc dài hạn và thiếu tính đối xứng trong xử lý đầu vào. Để khắc phục những điểm yếu này, một kiến trúc mạng nơ-ron mới ra đời, có khả năng xử lý các chuỗi đầu vào có độ dài bất kỳ và chia sẻ trọng số một cách nhất quán qua các bước thời gian – đó là **Mạng nơ-ron Hồi quy (Recurrent Neural Network – RNN)**. Phần này sẽ đi sâu vào kiến trúc, cơ chế huấn luyện, và các tiêu chí đánh giá của RNN, đồng thời thảo luận về những ưu điểm và thách thức của chúng trong ứng dụng thực tế.

2.1 Từ Mô hình Cửa sổ Cố định đến RNN

Mô hình ngôn ngữ nơ-ron với cửa sổ cố định (fixed-window neural LM) [?, p.22] đã giải quyết được vấn đề thưa thớt của n-gram bằng cách sử dụng các vector từ liên tục. Tuy nhiên, nó vẫn còn hai nhược điểm lớn:

- Kích thước cửa sổ là cố định; việc mở rộng cửa sổ làm tăng kích thước ma trận trọng số và không bao giờ đủ lớn để bao quát toàn bộ ngữ cảnh.
- Các từ ở các vị trí khác nhau trong cửa sổ được xử lý bằng các trọng số khác nhau, dẫn đến thiếu tính đối xứng và không tận dụng được cấu trúc tuần tự của ngôn ngữ.

RNN giải quyết cả hai vấn đề này bằng cách áp dụng *cùng một bộ trọng số* cho mọi bước thời gian, đồng thời duy trì một trạng thái ẩn (hidden state) được cập nhật tuần tự, cho phép mô hình "ghi nhớ" thông tin từ tất cả các bước trước đó trong chuỗi [?, p.23].

2.2 Kiến trúc RNN

Tại mỗi bước thời gian t , RNN nhận đầu vào là từ hiện tại $x^{(t)}$ (được biểu diễn dưới dạng vector one-hot hoặc embedding) và trạng thái ẩn từ bước trước $h^{(t-1)}$, đồng thời tạo ra trạng thái ẩn mới $h^{(t)}$ và một phân phối xác suất trên từ vựng $\hat{y}^{(t)}$. Các phương trình cốt lõi của RNN như sau [?, p.24]:

$$e^{(t)} = Ex^{(t)} \quad (\text{word embedding}) \quad (1)$$

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right) \quad (\text{hidden state}) \quad (2)$$

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \quad (\text{output distribution}) \quad (3)$$

Trong đó:

- $E \in \mathbb{R}^{d \times |V|}$: ma trận embedding, biến đổi vector one-hot $x^{(t)}$ thành vector embedding $e^{(t)}$.
- $W_e \in \mathbb{R}^{D_h \times d}$: ma trận trọng số kết nối từ embedding đến tầng ẩn.
- $W_h \in \mathbb{R}^{D_h \times D_h}$: ma trận trọng số lặp (recurrent weight), kết nối trạng thái ẩn trước đó với trạng thái hiện tại.
- $b_1 \in \mathbb{R}^{D_h}$: bias cho tầng ẩn.
- $U \in \mathbb{R}^{|V| \times D_h}$, $b_2 \in \mathbb{R}^{|V|}$: ma trận và bias cho tầng đầu ra.
- $\sigma(\cdot)$: hàm kích hoạt phi tuyến (thường là tanh hoặc ReLU).

Điểm quan trọng là các ma trận W_e, W_h, U được *chia sẻ* (shared) qua mọi bước thời gian. Điều này có nghĩa là số lượng tham số của mô hình không phụ thuộc vào độ dài của chuỗi đầu vào, và các từ ở các vị trí khác nhau được xử lý một cách đối xứng – một bước tiến lớn so với mô hình cửa sổ cố định [?, p.25].

2.3 Huấn luyện RNN: Backpropagation Through Time

RNN được huấn luyện bằng cách tối thiểu hóa hàm mất mát cross-entropy trên toàn bộ chuỗi. Tại mỗi bước thời gian t , mất mát là âm log-likelihood của từ thực tế $x^{(t+1)}$:

$$J^{(t)}(\theta) = -\log \hat{y}_{x_{t+1}}^{(t)}, \quad (4)$$

và tổng mất mát trung bình trên một chuỗi (hoặc một batch) là [?, p.26]:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log \hat{y}_{x_{t+1}}^{(t)}. \quad (5)$$

Để tính gradient của J đối với các trọng số (đặc biệt là ma trận lặp W_h), chúng ta sử dụng thuật toán **Lan truyền ngược qua thời gian** (Backpropagation Through Time – BPTT) [?, p.33]. BPTT hoạt động bằng cách mở rộng RNN theo thời gian thành một mạng nơ-ron truyền thẳng (feed-forward) có độ sâu bằng chiều dài chuỗi, sau đó áp dụng quy tắc chuỗi thông thường. Vì W_h xuất hiện ở mọi bước thời gian, gradient của nó được tính bằng tổng các gradient cục bộ tại mỗi bước [?, p.33]:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}, \quad (6)$$

trong đó $\left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$ là gradient đóng góp từ bước thời gian i khi W_h được sử dụng để tạo ra $h^{(i)}$. Phép cộng dồn này phản ánh quy tắc chuỗi đa biến và là cốt lõi của BPTT [?, p.35].

Trong thực tế, thay vì duyệt toàn bộ ngữ liệu, chúng ta sử dụng SGD với các batch nhỏ (thường là các câu hoặc đoạn văn bản) để cập nhật trọng số [?, p.32].

2.4 Đánh giá Mô hình ngôn ngữ: Perplexity

Độ rối (Perplexity) là chỉ số đánh giá tiêu chuẩn cho các mô hình ngôn ngữ. Perplexity của một mô hình trên một đoạn văn bản $x^{(1)}, \dots, x^{(T)}$ được định nghĩa là nghịch đảo của xác suất trung bình hình học của toàn bộ văn bản [?, p.39]:

$$\text{Perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(x^{(t+1)} \mid x^{(1)}, \dots, x^{(t)})} \right)^{1/T}. \quad (7)$$

Chỉ số này tương đương với mũ của hàm mất mát cross-entropy trung bình:

$$\text{Perplexity} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log \hat{y}_{x_{t+1}}^{(t)} \right) = \exp(J(\theta)). \quad (8)$$

Một perplexity thấp hơn cho thấy mô hình tự tin hơn và chính xác hơn trong dự đoán từ tiếp theo. Ví dụ, perplexity bằng 100 có nghĩa là mô hình trung bình đang phân vân giữa 100 lựa chọn từ vựng cho từ tiếp theo.

2.5 Ưu điểm và Nhược điểm của RNN

RNN mang lại nhiều lợi thế so với các mô hình ngôn ngữ truyền thống [?, p.25]:

- **Xử lý chuỗi độ dài bất kỳ:** RNN có thể làm việc với các chuỗi đầu vào có độ dài biến đổi, không bị giới hạn bởi cửa sổ cố định.
- **Không tăng kích thước mô hình theo độ dài:** Các tham số được chia sẻ qua thời gian, vì vậy số lượng tham số không phụ thuộc vào chiều dài của văn bản.
- **Sử dụng thông tin từ các bước xa:** Về lý thuyết, trạng thái ẩn $h^{(t)}$ có thể mang thông tin từ toàn bộ lịch sử, cho phép mô hình nắm bắt các phụ thuộc dài hạn.
- **Tính đối xứng:** Cùng một bộ trọng số áp dụng cho mọi vị trí, phù hợp với tính chất tuần tự của ngôn ngữ.

Tuy nhiên, RNN cũng có hai nhược điểm chính [?, p.25]:

- **Tốc độ tính toán chậm:** Do bản chất tuần tự, RNN không thể tính toán song song các bước thời gian, dẫn đến thời gian huấn luyện và suy luận kéo dài.
- **Khó học phụ thuộc xa trong thực tế:** Mặc dù lý thuyết cho phép, nhưng trên thực tế, các RNN cơ bản gặp khó khăn trong việc học các phụ thuộc ở khoảng cách lớn do các vấn đề về biến mất và bùng nổ gradient, sẽ được thảo luận chi tiết trong phần tiếp theo [?, p.40].

Những thách thức này đã thúc đẩy sự ra đời của các kiến trúc cải tiến như LSTM và GRU, cũng như các kỹ thuật cắt gradient và khởi tạo đặc biệt, giúp RNN trở thành một công cụ mạnh mẽ và linh hoạt trong NLP hiện đại.

3 Các vấn đề cốt lõi của RNN: Bùng nổ và Biến mất Gradients

Mặc dù RNN có tiềm năng lý thuyết để học các phụ thuộc dài hạn trong chuỗi văn bản, trên thực tế, việc huấn luyện các RNN cơ bản gặp phải hai vấn đề nghiêm trọng liên quan đến sự lan truyền của gradient: **biến mất gradient (vanishing gradient)** và **bùng nổ gradient (exploding gradient)** [?, p.40]. Hai hiện tượng này khiến mô hình khó có thể nắm bắt được thông tin từ các bước thời gian ở xa, dẫn đến hiệu suất kém trên các tác vụ yêu cầu ngữ cảnh dài. Phần này sẽ giải thích nguyên nhân toán học của các vấn đề này, hậu quả của chúng đối với quá trình huấn luyện, và các kỹ thuật thực tế để giảm thiểu chúng.

3.1 Hiện tượng Biến mất Gradient (Vanishing Gradient)

Trong quá trình lan truyền ngược qua thời gian (BPTT), gradient của hàm mất mát tại bước t đối với trọng số lặp W_h được tính là tổng của các đóng góp từ các bước trước đó [?, p.35]:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial W_h} \Big|_{(i)}.$$

Mỗi đóng góp $\frac{\partial J^{(t)}}{\partial W_h} \Big|_{(i)}$ liên quan đến tích của các ma trận Jacobian của trạng thái ẩn qua các bước thời gian trung gian:

$$\frac{\partial J^{(t)}}{\partial W_h} \Big|_{(i)} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \left(\prod_{j=i+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \frac{\partial h^{(i)}}{\partial W_h}.$$

Với $\frac{\partial h^{(j)}}{\partial h^{(j-1)}} = W_h^T \text{diag}(f'(h^{(j-1)}))$, trong đó f là hàm kích hoạt (thường là sigmoid hoặc tanh). Nếu chuẩn của ma trận Jacobian này nhỏ hơn 1, thì tích của chúng theo cấp số mũ sẽ tiến về 0 khi khoảng cách $t - i$ lớn [?, p.6]:

$$\left\| \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right\| \leq \|W_h^T\| \cdot \|\text{diag}(f'(h^{(j-1)}))\| \leq \beta_W \beta_h,$$

và do đó:

$$\left\| \frac{\partial h^{(t)}}{\partial h^{(i)}} \right\| \leq (\beta_W \beta_h)^{t-i}.$$

Nếu $\beta_W \beta_h < 1$, gradient sẽ biến mất theo cấp số nhân khi $t - i$ tăng [?, p.7]. Điều này giải thích tại sao RNN cơ bản không thể học được các phụ thuộc ngôn ngữ ở khoảng cách xa, như ví dụ sau [?, p.47]:

"When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her -----."

Để dự đoán từ "tickets" ở cuối câu, mô hình cần kết nối với từ "tickets" xuất hiện cách đó 7 bước. Tuy nhiên, gradient từ bước đó bị suy yếu nghiêm trọng, khiến mô hình không thể học được mối quan hệ này.

3.2 Hiện tượng Bùng nổ Gradient (Exploding Gradient)

Ngược lại, nếu $\beta_W \beta_h > 1$, gradient sẽ tăng lên theo cấp số nhân theo thời gian, dẫn đến hiện tượng bùng nổ [?, p.7]. Khi đó, bước cập nhật SGD trở nên quá lớn, có thể đẩy các tham số đến các vùng không gian tham số tồi tệ, gây ra sự bất ổn và thậm chí làm cho các giá trị trở nên vô cực (NaN) [?, p.48]. Điều này thường xuất hiện dưới dạng các giá trị mất mát đột ngột tăng vọt trong quá trình huấn luyện.

3.3 Giải pháp cho Bùng nổ Gradient: Gradient Clipping

May mắn thay, vấn đề bùng nổ gradient có một giải pháp đơn giản và hiệu quả: **cắt gradient (gradient clipping)** [?, p.49]. Ý tưởng là trước khi thực hiện cập nhật SGD, nếu chuẩn của gradient $\hat{g}$ vượt quá một ngưỡng t xác định, ta sẽ thu nhỏ gradient đó về ngưỡng:

$$\text{if } \|\hat{g}\| \geq t, \quad \hat{g} \leftarrow \frac{t}{\|\hat{g}\|} \hat{g}.$$

Kỹ thuật này đảm bảo bước cập nhật không quá lớn, giữ cho quá trình huấn luyện ổn định mà vẫn giữ nguyên hướng của gradient [?, p.49].

3.4 Giải pháp cho Biến mất Gradient

Vấn đề vanishing gradient phức tạp hơn và đòi hỏi các chiến lược sâu hơn:

- **Khởi tạo ma trận trọng số:** Khởi tạo W_h dưới dạng ma trận đơn vị (identity matrix) hoặc sử dụng các phương pháp khởi tạo thông minh (như Xavier) có thể giúp duy trì gradient qua các bước thời gian [?, p.8].
- **Sử dụng hàm kích hoạt ReLU:** ReLU có đạo hàm bằng 1 đối với các giá trị dương, cho phép gradient truyền qua mà không bị suy giảm, giảm thiểu hiệu ứng vanishing [?, p.8].
- **Kiến trúc có cổng (Gated Architectures):** Đây là giải pháp mạnh mẽ nhất và được sử dụng rộng rãi nhất. Các mô hình như **Long Short-Term Memory (LSTM)** và **Gated Recurrent Units (GRU)** được thiết kế với các cơ chế cổng (forget gate, input gate, output gate) cho phép kiểm soát luồng thông tin qua các bước thời gian, tạo ra các đường kết nối tuyến tính trực tiếp giúp gradient lan truyền dễ dàng hơn trong quá trình ngược [?, p.50]. Những kiến trúc này sẽ được giới thiệu chi tiết trong phần tiếp theo.

Tóm lại, vanishing và exploding gradient là hai trở ngại lớn đối với việc huấn luyện RNN cơ bản trên chuỗi dài. Trong khi gradient clipping là giải pháp đơn giản cho exploding, việc giải quyết vanishing gradient đòi hỏi các cải tiến sâu hơn về cả khởi tạo, hàm kích hoạt, và đặc biệt là kiến trúc mạng, mở đường cho các mô hình mạnh mẽ hơn như LSTM và GRU.

4 Cấu trúc mạng cải tiến: GRU, LSTM và Mạng Đa chiều/Sâu

Phần trước đã chỉ ra rằng các RNN cơ bản gặp khó khăn trong việc học các phụ thuộc dài hạn do hiện tượng biến mất gradient. Để giải quyết vấn đề này, các kiến trúc mạng nơ-ron với cơ chế cổng (gated architectures) đã được đề xuất, cho phép mô hình kiểm soát luồng thông tin qua các bước thời gian một cách linh hoạt hơn. Hai kiến trúc tiêu biểu nhất là **Gated Recurrent Unit (GRU)** và **Long Short-Term Memory (LSTM)**, cả hai đều tạo ra các đường kết nối tuyến tính trực tiếp, giúp gradient lan truyền dễ dàng hơn trong quá trình lan truyền ngược [?, p.50]. Ngoài ra, để tăng cường khả năng biểu diễn, các biến thể như mạng RNN đa chiều (bidirectional RNN) và mạng RNN sâu (deep RNN) cũng được phát triển. Phần này sẽ trình bày chi tiết về các kiến trúc này và trực giác đằng sau chúng.

4.1 Gated Recurrent Unit (GRU)

GRU, được giới thiệu bởi Cho et al. (2014), là một kiến trúc đơn giản hơn so với LSTM nhưng vẫn đạt hiệu quả cao trong nhiều tác vụ. GRU sử dụng hai cơ chế cổng: **cổng cập nhật (update gate)** và **cổng đặt lại (reset gate)**, để điều chỉnh luồng thông tin [?, p.11]. Các phương trình cốt lõi của GRU như sau:

$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1}) \quad (\text{update gate})$$

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1}) \quad (\text{reset gate})$$

$$\tilde{h}_t = \tanh(r_t \circ U h_{t-1} + W x_t) \quad (\text{new memory})$$

$$h_t = (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1} \quad (\text{hidden state})$$

Trong đó $\circ$ là tích Hadamard (nhân từng phần tử). Trực giác của GRU có thể được hiểu qua bốn giai đoạn [?, p.12]:

1. **Tạo bộ nhớ mới ($\tilde{h}_t$):** Kết hợp đầu vào hiện tại x_t với trạng thái ẩn trước đó h_{t-1} để tạo ra một ứng viên cho bộ nhớ mới.
2. **Cổng đặt lại (r_t):** Xác định mức độ ảnh hưởng của h_{t-1} lên bộ nhớ mới. Nếu $r_t \approx 0$, mô hình hoàn toàn bỏ qua quá khứ và chỉ tập trung vào đầu vào hiện tại.
3. **Cổng cập nhật (z_t):** Quyết định bao nhiêu phần trăm của bộ nhớ cũ được giữ lại và bao nhiêu phần trăm được thay thế bằng bộ nhớ mới. Khi $z_t \approx 1$, phần lớn trạng thái ẩn cũ được sao chép sang bước tiếp theo, giúp duy trì thông tin dài hạn.

4. **Trạng thái ẩn cuối cùng (h_t):** Là sự kết hợp có trọng số giữa bộ nhớ cũ và bộ nhớ mới theo chỉ dẫn của cổng cập nhật.

Nhờ cơ chế cổng, GRU có thể học cách giữ lại thông tin quan trọng trong nhiều bước thời gian và loại bỏ thông tin không cần thiết, giải quyết hiệu quả vấn đề vanishing gradient.

4.2 Long Short-Term Memory (LSTM)

LSTM, được đề xuất bởi Hochreiter và Schmidhuber (1997), là một trong những kiến trúc RNN thành công và được sử dụng rộng rãi nhất [?, p.13]. LSTM có cấu trúc phức tạp hơn GRU với ba cổng điều khiển và một bộ nhớ tế bào (cell memory) riêng biệt. Các phương trình của LSTM như sau:

$$\begin{aligned} i_t &= \sigma(W^{(i)}x_t + U^{(i)}h_{t-1}) && \text{(input gate)} \\ f_t &= \sigma(W^{(f)}x_t + U^{(f)}h_{t-1}) && \text{(forget gate)} \\ o_t &= \sigma(W^{(o)}x_t + U^{(o)}h_{t-1}) && \text{(output gate)} \\ \tilde{c}_t &= \tanh(W^{(c)}x_t + U^{(c)}h_{t-1}) && \text{(new memory cell)} \\ c_t &= f_t \circ c_{t-1} + i_t \circ \tilde{c}_t && \text{(final memory cell)} \\ h_t &= o_t \circ \tanh(c_t) && \text{(hidden state)} \end{aligned}$$

Trực giác của LSTM có thể được hiểu qua năm giai đoạn [?, p.14]:

1. **Tạo bộ nhớ mới ($\tilde{c}_t$):** Tương tự như GRU, tạo ra một ứng viên bộ nhớ mới từ x_t và h_{t-1} .
2. **Cổng đầu vào (i_t):** Đánh giá mức độ quan trọng của từ mới và quyết định bao nhiêu phần trăm của $\tilde{c}_t$ sẽ được thêm vào bộ nhớ tế bào.
3. **Cổng quên (f_t):** Xác định bao nhiêu phần trăm của bộ nhớ tế bào cũ c_{t-1} sẽ bị loại bỏ. Điều này cho phép LSTM "quên" đi những thông tin không còn hữu ích.
4. **Bộ nhớ tế bào cuối cùng (c_t):** Kết hợp bộ nhớ cũ đã được "quên" với bộ nhớ mới đã được "gating" để tạo ra bộ nhớ hiện tại.
5. **Cổng đầu ra (o_t) và trạng thái ẩn (h_t):** Cổng đầu ra kiểm soát phần nào của bộ nhớ tế bào được tiết lộ dưới dạng trạng thái ẩn. Điều này cho phép tách biệt bộ nhớ nội tại khỏi trạng thái được sử dụng trong các phép tính tiếp theo.

Sự khác biệt chính giữa LSTM và GRU là LSTM có bộ nhớ tế bào riêng (c_t) được duy trì độc lập với trạng thái ẩn (h_t), và có cổng quên và cổng đầu ra riêng biệt, mang lại nhiều khả năng kiểm soát hơn. Trong thực tế, cả hai đều hoạt động tốt, nhưng LSTM thường được ưa chuộng trong các tác vụ yêu cầu bộ nhớ dài hạn phức tạp [?, p.13].

4.3 Mạng RNN Đa chiều (Bidirectional RNN)

Trong nhiều tác vụ NLP, việc sử dụng thông tin từ *cả hai phía* của một từ có thể cải thiện đáng kể hiệu suất. Ví dụ, trong nhận dạng thực thể có tên, từ phía sau cũng cung cấp ngữ cảnh quan trọng. **Mạng RNN đa chiều (Bidirectional RNN)** giải quyết vấn đề này bằng cách chạy hai luồng RNN song song: một luồng đọc từ trái sang phải ($\vec{h}_t$) và một luồng đọc từ phải sang trái ($\overleftarrow{h}_t$) [?, p.9]. Trạng thái ẩn cuối cùng tại mỗi bước t là sự ghép nối (concatenation) của hai trạng thái:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Các phương trình của bidirectional RNN [?, p.9]:

$$\vec{h}_t = f(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b}) \quad (9)$$

$$\overleftarrow{h}_t = f(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h}_{t+1} + \overleftarrow{b}) \quad (10)$$

$$\hat{y}_t = g(Uh_t + c) = g(U[\vec{h}_t; \overleftarrow{h}_t] + c). \quad (11)$$

Bidirectional RNN thường được sử dụng trong các mô hình encoder (ví dụ: cho dịch máy, phân tích cảm xúc) vì nó có thể tận dụng toàn bộ ngữ cảnh của đầu vào.

4.4 Mạng RNN Sâu (Deep RNN)

Để tăng cường khả năng biểu diễn của mô hình, ta có thể xếp chồng nhiều lớp RNN lên nhau, tạo thành **mạng RNN sâu (Deep RNN)** [?, p.9]. Trong kiến trúc này, đầu ra của lớp RNN thứ $l - 1$ tại bước t được sử dụng làm đầu vào cho lớp RNN thứ l tại cùng bước thời gian [?, p.9]. Phương trình tổng quát cho lớp i :

$$\vec{h}_t^{(i)} = f(\vec{W}^{(i)} h_t^{(i-1)} + \vec{V}^{(i)} \vec{h}_{t-1}^{(i)} + \vec{b}^{(i)}),$$

và tương tự cho chiều ngược. Đầu ra cuối cùng của mạng là sự ghép nối của hai trạng thái tại lớp cuối cùng:

$$\hat{y}_t = g(U[\vec{h}_t^{(L)}; \overleftarrow{h}_t^{(L)}] + c).$$

Deep RNN có thể học các biểu diễn ngày càng trừu tượng qua các lớp, nhưng cũng đòi hỏi nhiều dữ liệu huấn luyện hơn và có nguy cơ bị quá khớp cao hơn [?, p.10].

Tóm lại, các kiến trúc cải tiến như GRU, LSTM, bidirectional RNN và deep RNN đã giúp vượt qua nhiều hạn chế của RNN cơ bản, đặc biệt là trong việc xử lý các phụ thuộc dài hạn và tận dụng ngữ cảnh hai chiều. Những kiến trúc này là nền tảng cho các mô hình mạnh mẽ hơn như seq2seq và Transformer, sẽ được giới thiệu trong các bài giảng tiếp theo.

5 Dịch máy và Mô hình Sequence-to-Sequence

Mô hình ngôn ngữ và RNN không chỉ là những công cụ học thuật mà còn là nền tảng cho một trong những ứng dụng thành công nhất của học sâu trong NLP: **Dịch máy Nơ-ron (Neural Machine Translation – NMT)** [?, p.51]. NMT đã thay thế hoàn toàn các hệ thống dịch máy thống kê truyền thống (Statistical Machine Translation – SMT) chỉ trong vòng vài năm, từ một hướng nghiên cứu thử nghiệm vào năm 2014 trở thành phương pháp tiêu chuẩn vào năm 2016, khi Google Translate chuyển đổi sang NMT [?, p.52]. Thành công này được xây dựng trên kiến trúc **Sequence-to-Sequence (Seq2Seq)** với mô hình Encoder-Decoder, một khuôn khổ linh hoạt có thể áp dụng cho nhiều tác vụ NLP khác như tóm tắt văn bản, đối thoại, phân tích cú pháp, và sinh mã lập trình [?, p.54]. Phần này sẽ trình bày kiến trúc Seq2Seq, cơ chế huấn luyện, và thách thức cốt lõi của nó – bài toán nút thắt cổ chai (bottleneck).

5.1 NMT như một Mô hình Ngôn ngữ Có điều kiện

Dịch máy là nhiệm vụ dịch một câu x từ ngôn ngữ nguồn (source) sang câu y trong ngôn ngữ đích (target) [?, p.51]. Mô hình Seq2Seq tiếp cận bài toán này bằng cách tính trực tiếp xác suất của câu đích khi biết câu nguồn [?, p.55]:

$$P(y \mid x) = P(y_1 \mid x) \cdot P(y_2 \mid y_1, x) \cdot \dots \cdot P(y_T \mid y_1, \dots, y_{T-1}, x).$$

Đây là một **Mô hình Ngôn ngữ Có điều kiện (Conditional Language Model)**, bởi vì bộ giải mã (decoder) dự đoán từ tiếp theo của câu đích dựa trên các từ đã sinh ra trước đó, nhưng đồng thời cũng bị *điều kiện hóa* (conditioned) bởi câu nguồn x [?, p.55].

5.2 Kiến trúc Encoder-Decoder

Mô hình Seq2Seq bao gồm hai thành phần chính [?, p.53]:

1. **Encoder (Bộ mã hóa)**: Một mạng RNN (thường là LSTM hoặc GRU, có thể là bidirectional hoặc deep) đọc toàn bộ câu nguồn $x = (x_1, \dots, x_n)$ theo thứ tự và tạo ra một **vector ngữ cảnh (context vector)** c , là trạng thái ẩn cuối cùng của encoder [?, p.57]:

$$c = h_n,$$

hoặc một tổ hợp của các trạng thái ẩn. Vector này được kỳ vọng sẽ tóm tắt toàn bộ ý nghĩa của câu nguồn.

2. **Decoder (Bộ giải mã):** Một mạng RNN khác (có thể là LSTM hoặc GRU) nhận vector ngữ cảnh c làm trạng thái ẩn khởi tạo (hoặc làm đầu vào bổ sung), và sinh ra câu đích $y = (y_1, \dots, y_m)$ từng từ một. Tại mỗi bước thời gian, decoder dự đoán phân phối xác suất của từ tiếp theo dựa trên vector ngữ cảnh, trạng thái ẩn trước đó, và từ đã sinh ra trước đó [?, p.55]:

$$P(y_t | y_1, \dots, y_{t-1}, x) = \text{softmax}(U h_t^{(dec)}).$$

Mô hình này được huấn luyện **end-to-end** trên các cặp câu song ngữ (parallel corpus), sử dụng thuật toán lan truyền ngược để tối ưu hóa toàn bộ hệ thống [?, p.56].

5.3 Mở rộng và Cải tiến cho Seq2Seq

Các phiên bản thực tế của Seq2Seq thường áp dụng nhiều cải tiến để cải thiện chất lượng dịch [?, p.10]:

- **Trọng số riêng biệt cho Encoder và Decoder:** Sử dụng các ma trận RNN khác nhau cho encoder và decoder, thay vì dùng chung, cho phép mỗi module tối ưu hóa cho vai trò riêng [?, p.10].
- **Nhiều đầu vào cho Decoder:** Thay vì chỉ nhận trạng thái ẩn trước đó, mỗi bước của decoder có thể nhận ba đầu vào: trạng thái ẩn trước đó h_{t-1} , vector ngữ cảnh c (từ encoder), và từ vừa được dự đoán $\hat{y}_{t-1}$ [?, p.10]:

$$h_t = \phi(h_{t-1}, c, y_{t-1}).$$

- **Encoder đa chiều (Bidirectional Encoder):** Sử dụng bidirectional RNN cho encoder để tận dụng ngữ cảnh từ cả hai phía của câu nguồn, cải thiện chất lượng vector ngữ cảnh [?, p.10].
- **Mạng sâu (Deep RNN):** Xếp chồng nhiều lớp RNN trong cả encoder và decoder để tăng khả năng học biểu diễn [?, p.10].
- **Đảo ngược câu nguồn (Reversing Source Sentence):** Đảo tạo mô hình với câu nguồn bị đảo ngược (ví dụ: dịch $CBA \rightarrow XY$ thay vì $ABC \rightarrow XY$) để rút ngắn khoảng cách giữa các từ tương ứng, giảm tác động của vanishing gradient [?, p.10].

5.4 Vấn đề Nút thắt Cổ chai (Bottleneck Problem)

Mặc dù Seq2Seq đã đạt được thành công vang dội, nó vẫn tồn tại một nhược điểm cố hữu: **bài toán nút thắt cổ chai (bottleneck problem)** [?, p.58]. Toàn bộ thông tin của câu nguồn phải được nén vào một vector ngữ cảnh c có độ dài cố định (thường là chiều của trạng thái ẩn, ví dụ 256 hoặc 512). Đối với các câu dài (hơn 30-40 từ), vector này không đủ dung lượng để lưu trữ toàn bộ ý nghĩa, dẫn đến việc decoder mất thông tin quan trọng [?, p.57].

Vấn đề này đã thúc đẩy sự ra đời của **cơ chế Chú ý (Attention)**, sẽ được giới thiệu trong bài giảng tiếp theo, cho phép decoder "nhìn" trực tiếp vào các trạng thái ẩn của encoder tại mỗi bước thời gian, thay vì chỉ dựa vào một vector cố định. Attention đã trở thành một thành phần không thể thiếu trong các mô hình NLP hiện đại và là nền tảng cho kiến trúc Transformer, xương sống của các LLM ngày nay.

Tóm lại, Seq2Seq với kiến trúc Encoder-Decoder đã cách mạng hóa dịch máy và nhiều tác vụ NLP khác bằng cách học trực tiếp ánh xạ từ chuỗi đầu vào sang chuỗi đầu ra. Dù còn tồn tại hạn chế về nút thắt cổ chai, đây là một bước tiến quan trọng, đặt nền móng cho các mô hình mạnh mẽ hơn dựa trên cơ chế chú ý và Transformer.

6 Tổng kết

Trong bài giảng này, chúng ta đã cùng nhau khám phá một trong những khái niệm nền tảng và quan trọng nhất của Xử lý Ngôn ngữ Tự nhiên hiện đại: **Mô hình Ngôn ngữ (Language Modeling)**. Hành trình bắt đầu từ việc hiểu rõ định nghĩa và tầm quan trọng của bài toán dự đoán từ tiếp theo – một nhiệm vụ tưởng chừng đơn giản nhưng lại là cốt lõi của hầu hết các hệ thống trí tuệ nhân tạo tạo sinh văn bản ngày nay, từ gợi ý văn bản trên điện thoại cho đến các mô hình tiên tiến như ChatGPT [?, p.3].

Chúng ta đã thấy sự tiến hóa của các phương pháp xây dựng mô hình ngôn ngữ, từ các mô hình **n-gram** truyền thống dựa trên thống kê tần suất [?, p.9], với những hạn chế cố hữu về tính thưa thớt (sparsity), yêu cầu lưu trữ lớn, và không thể nắm bắt các phụ thuộc xa trong văn bản [?, p.12]. Mô hình nơ-ron với cửa sổ cố định [?, p.22] đã giải quyết được vấn đề thưa thớt nhưng vẫn bị giới hạn bởi độ dài ngữ cảnh và thiếu tính đối xứng trong xử lý.

Bước đột phá thực sự đến với **Mạng nơ-ron Hồi quy (RNN)**, một kiến trúc có khả năng xử lý các chuỗi đầu vào có độ dài bất kỳ thông qua việc chia sẻ trọng số qua các bước thời gian [?, p.23]. Chúng ta đã tìm hiểu cơ chế hoạt động của RNN, cách huấn luyện chúng thông qua thuật toán **Lan truyền ngược qua thời gian (BPTT)** [?, p.33], và chỉ số đánh giá **Perplexity** [?, p.39]. RNN mang lại nhiều ưu điểm vượt trội, nhưng cũng đối mặt với hai thách thức lớn: **hiện tượng biến mất và bùng nổ gradient** [?, p.40]. Những hiện tượng này khiến mô hình khó học được các phụ thuộc ngôn ngữ ở khoảng cách xa, làm suy giảm hiệu suất trên các văn bản dài.

Để khắc phục những hạn chế này, các kiến trúc mạng với cơ chế cổng (gated architectures) đã ra đời, tiêu biểu là **LSTM** và **GRU** [?, p.13]. Nhờ các cổng điều khiển (cổng quên, cổng đầu vào, cổng đầu ra), các mô hình này có thể duy trì và cập nhật bộ nhớ một cách linh hoạt qua nhiều bước thời gian, giải quyết hiệu quả vấn đề vanishing gradient [?, p.14]. Chúng ta cũng đã tìm hiểu các biến thể mở rộng của RNN như **mạng đa chiều (bidirectional RNN)** cho phép tận dụng ngữ cảnh hai chiều, và **mạng sâu (deep RNN)** giúp tăng cường khả năng biểu diễn [?, p.9].

Cuối cùng, chúng ta đã thấy sức mạnh ứng dụng của RNN thông qua **Dịch máy Nơ-ron (NMT)** với kiến trúc **Sequence-to-Sequence (Encoder-Decoder)** [?, p.51]. Mô hình này đã thay thế hoàn toàn các hệ thống dịch máy thống kê truyền thống chỉ trong vài năm, khẳng định sức mạnh của học sâu trong NLP [?, p.52]. Dù còn tồn tại vấn đề nút thắt cổ chai (bottleneck) – khi toàn bộ thông tin của câu nguồn bị nén vào một vector ngữ cảnh cố định [?, p.58] – Seq2Seq đã đặt nền móng cho các mô hình tiên tiến hơn, đặc biệt là cơ chế **Chú ý (Attention)** và kiến trúc **Transformer**, sẽ được khám phá trong các bài giảng tiếp theo.

Như vậy, chúng ta đã hoàn thành một vòng tròn kiến thức từ mô hình ngôn ngữ cơ bản đến các mạng RNN và ứng dụng của chúng. Đây là những nền tảng quan trọng để hiểu được cách các mô hình ngôn ngữ lớn (LLMs) hiện đại hoạt động, cũng như cách áp dụng chúng vào các bài toán NLP thực tế.

References